

Học Sâu (Deep Learning)

Chương 9: Các Mẫu Kiến trúc Mạng Tích chập (ConvNet Architecture Patterns)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Nguyên lý Thiết kế Kiến trúc
2. Kết nối Thặng dư (Residual Connections)
3. Batch Normalization (Chuẩn hóa Hàng loạt)
4. Tích chập Tách rời (Separable Convolution)
5. Thực hành: Xây dựng Mini-Xception
6. Kỹ nguyên mới: Vision Transformers (ViTs)
7. Tổng kết

Không gian Giả thuyết (Hypothesis Space)

- **Kiến trúc Mô hình (Model Architecture)** là nghệ thuật lắp ráp các tầng mạng (Layers).
- Cấu trúc này định nghĩa **Không gian Giả thuyết** (Hypothesis Space). Thuật toán Gradient Descent sẽ tìm kiếm bộ tham số (Weights) tối ưu bên trong không gian này.
- Một kiến trúc tốt giống như một bản thiết kế thông minh, giúp đóng gói **Tri thức tiên nghiệm (Prior Knowledge)** của kỹ sư vào mô hình.
- Ví dụ: Việc sử dụng Tích chập (Convolution) thay cho mạng Dense hàm chứa tri thức tiên nghiệm rằng: *"Các mẫu cục bộ trong hình ảnh là bất biến với sự dịch chuyển"*.

Tầm quan trọng của Thiết kế Kiến trúc

- Thiết kế khéo léo sẽ làm **thu hẹp không gian tìm kiếm**, giúp Gradient Descent hội tụ nhanh hơn về điểm tối ưu.
- Nó cũng giúp mô hình sử dụng dữ liệu hiệu quả hơn, chống lại hiện tượng Overfitting ngay cả khi tập dữ liệu huấn luyện khá nhỏ.
- Ngược lại, nếu chọn một kiến trúc sai lầm (ví dụ: dùng mạng Dense để nhận diện ảnh 4K), mô hình sẽ mãi mắc kẹt ở hiệu suất thấp, và không có một lượng dữ liệu lớn nào có thể cứu vãn được.

Nghệ thuật Thiết kế vs. Bằng chứng Khoa học

- Thiết kế kiến trúc thường nghiêng về **Nghệ thuật (Art)** hơn là Khoa học chính xác (Science).
- Các kỹ sư máy học xuất chúng thường sử dụng **Trực giác (Intuition)** và kinh nghiệm nhận dạng mẫu (Pattern Matching) để ghép nối mô hình, thay vì dựa vào các chứng minh toán học khắt khe.
- Sự thật: Không ai có thể giải thích bằng toán học một cách hoàn hảo *tại sao* một kiến trúc lại hoạt động tốt hơn kiến trúc khác. Ta chỉ biết nó hiệu quả thông qua thực nghiệm.

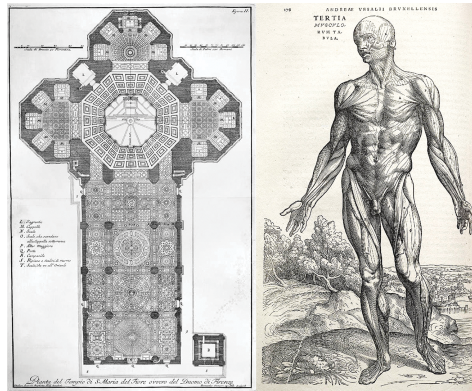
Ablation Studies (Nghiên cứu Cắt bỏ)

- Trong bối cảnh ngành Deep Learning có nhiều cấu trúc phức tạp được công bố chỉ để làm bài báo trông "nguy hiểm" hơn (Tối ưu hóa cho Peer Review).
- Mục tiêu thực sự của kỹ sư là **Tìm hiểu quan hệ Nhân quả**.
- **Ablation Studies:** Kỹ thuật gỡ bỏ từng thành phần của mô hình để kiểm chứng xem sự phức tạp được thêm vào có thực sự mang lại hiệu quả không.
- *Ví dụ:* Nếu mô hình $X + Y + Z$ hoạt động tốt, hãy thử chạy lại với chỉ X , Y , Z , $X+Y$... để biết chính xác sức mạnh đến từ đâu.

Công thức MHR: Modularity, Hierarchy, Reuse

Để quản lý một hệ thống phức tạp, ta dùng nguyên lý **MHR**:

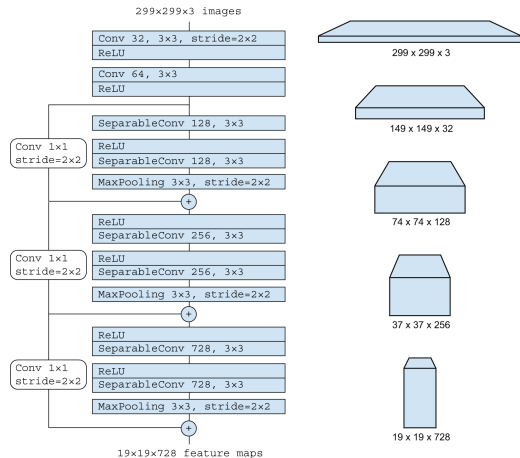
- **Modularity (Mô-đun hóa)**: Đóng gói các lớp tính toán cơ bản thành các Khối (Blocks) độc lập.
- **Hierarchy (Phân cấp)**: Xếp chồng các khối thành hệ thống hình Tháp (Pyramid).
- **Reuse (Tái sử dụng)**: Tái sử dụng cùng một cấu trúc khối lặp đi lặp lại.



Hình 9.1: Hệ thống phức tạp sinh học tuân theo nguyên lý MHR

Tháp Đặc trưng trong Mô hình Xception

- Các mạng CNN hiện đại (như Xception, ResNet) tuân thủ chặt chẽ nguyên lý MHR.
- Cấu trúc Lặp lại (Reuse): Khối $\text{SeparableConv} \rightarrow \text{SeparableConv} \rightarrow \text{MaxPooling}$.
- **Tháp không gian:** Kích thước bản đồ giảm dần (Shrinking: $299 \times 299 \rightarrow 19 \times 19$).
- **Tháp chiều sâu:** Số lượng bộ lọc (Filters) tăng dần (Deepening: $32 \rightarrow 728$).



Hình 9.2: Sự phân cấp của Khối đầu vào Xception

Vấn đề: Suy biến Đạo hàm (Vanishing Gradient)

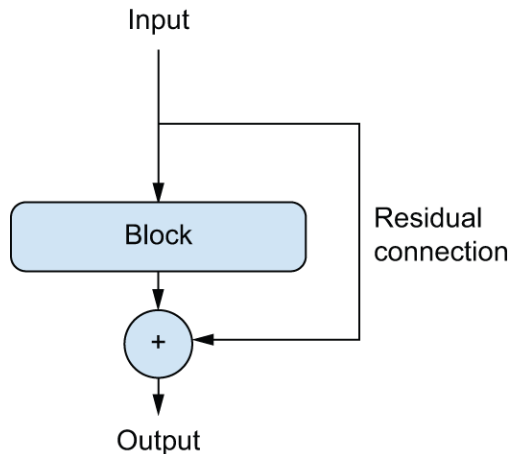
- Mạng càng **Sâu (Deep)**, hệ thống phân cấp (Hierarchy) càng hiệu quả vì nó ép mô hình tạo ra các đặc trưng trừu tượng cao cấp (Abstraction).
- Trực giác mạch bảo: Cứ xếp chồng thêm lớp, mô hình sẽ càng thông minh.
- Tuy nhiên, khi xếp chồng quá nhiều lớp (ví dụ: hàng trăm lớp Conv2D), mạng đột ngột **ngừng học**, Loss không suy giảm.
- Hiện tượng này không phải do Overfitting, mà do **Vanishing Gradient**.

Quy tắc Chuỗi (Chain Rule) và Trò chơi Truyền tin

- Thuật toán Backpropagation truyền đạo hàm ngược từ hàm Loss về các tầng đầu thông qua **Quy tắc chuỗi (Chain Rule)**.
- Quá trình này giống như trò chơi *Truyền tin tĩnh điện (Chinese Whispers/Telephone)*: Thông điệp qua từng người sẽ bị nhiễu dần.
- Phép nhân liên tiếp nhiều số nhỏ hơn 1 (qua các hàm ReLU, lớp Dropout) sẽ khiến đạo hàm giảm mạnh theo cấp số nhân về số 0.
- Các tầng đầu tiên không nhận được bất kỳ tín hiệu cập nhật nào, mô hình bị "tê liệt".

Giải pháp: Kết nối Thặng dư (Residual Connections)

- Phát minh của Kaiming He (2015) trong kiến trúc **ResNet** (Đạt giải Best Paper tại CVPR).
- Cung cấp một **đường tắt (Shortcut)** cho thông tin nhảy vọt qua khối tính toán (không bị biến đổi).
- Đầu vào được cộng trực tiếp vào đầu ra:
 $y = F(x) + x$.
- Đạo hàm chảy ngược xuyên qua các "đường cao tốc" này về đầu mạng một cách mượt mà.



Hình 9.3: Cơ chế hoạt động của Khối Residual

Mã nguồn giả (Pseudocode) cho Residual

Thay vì bắt khối (block) học ra một kết quả ánh xạ $F(x)$ hoàn toàn mới (rất khó), ta chỉ bắt nó học phần **thặng dư (Residual)** còn thiếu so với đầu vào.

```
1 # Some input tensor
2 x = ...
3 # Saves a reference to the original input. This is called the residual.
4 residual = x
5
6 # This computation block can potentially be destructive or noisy.
7 # It only needs to learn the "difference" (residual).
8 x = block(x)
9
10 # Adds the original input to the layer's output. The final output will
11 # thus always preserve full information about the original input.
12 x = add([x, residual])
```

Mã nguồn: Cân bằng số lượng Bộ lọc (Filters)

Phép cộng `add([x, residual])` đòi hỏi 2 Tensor phải có cùng hình dáng (Shape). Khi Khối Convolution làm tăng số lượng Channels, ta phải dùng một lớp Conv2D 1×1 để chiếu (Project) đường dẫn thẳng dư lên cùng số Channels.

```
1 inputs = keras.Input(shape=(32, 32, 3))
2 x = layers.Conv2D(32, 3, activation="relu")(inputs)
3 residual = x
4
5 # Block increases output filters from 32 to 64
6 x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
7
8 # Project the residual using 1x1 Conv2D to match 64 filters
9 residual = layers.Conv2D(64, 1)(residual)
10
11 x = layers.add([x, residual])
```

Mã nguồn: Cân bằng Kích thước (MaxPooling)

Tương tự, khi không gian bị thu nhỏ bởi MaxPooling2D, ta cũng phải thu nhỏ đường dẫn Residual bằng tham số `strides=2` để bảo đảm khớp kích thước khi cộng.

```
1 inputs = keras.Input(shape=(32, 32, 3))
2 x = layers.Conv2D(32, 3, activation="relu")(inputs)
3 residual = x
4
5 # Block includes MaxPooling downsampling
6 x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
7 x = layers.MaxPooling2D(2, padding="same")(x)
8
9 # Project the residual and apply downsampling with strides=2
10 residual = layers.Conv2D(64, 1, strides=2)(residual)
11
12 x = layers.add([x, residual])
```

Tại sao cần Chuẩn hóa Kích hoạt (Activation Normalization)?

- Chuẩn hóa dữ liệu đầu vào (căn giữa Mean=0, tỷ lệ Variance=1) giúp mô hình dễ hội tụ. (Ví dụ: scale pixel từ 0-255 về 0-1).
- Nhưng khi dữ liệu đi sâu vào các lớp bên trong, sự phân bố liên tục bị biến đổi sau mỗi phép nhân ma trận.
- Các tầng phía sau liên tục phải thích nghi với dữ liệu đầu vào mới thay đổi liên tục. Sự bất ổn này gọi là **Internal Covariate Shift**.
- **Batch Normalization** (Ioffe & Szegedy, 2015) giải quyết vấn đề bằng cách *chuẩn hóa lại dữ liệu ngay trong lúc đang di chuyển* giữa các tầng.